

Codebase Understanding Answers

This document completes the Myntra-style-store codebase-understanding prompts with a technical summary of the current React + Express stack.

1. Entry Point Architecture

The front-end entry begins in 'frontend/index.html', where Vite serves the HTML shell and injects the browser entry module 'src/main.jsx'. That file imports 'StrictMode' from React, 'createRoot' from 'react-dom/client', global styles from 'index.css', and the root component from 'App.jsx'. It mounts the app into the '#root' DOM node with 'createRoot(...).render(...)', so Vite is responsible for serving the module graph in development while React takes over rendering once the page loads.

'frontend/src/App.jsx' is the application root component. It owns the major UI sections, initializes page state, fetches catalog data from the backend, derives category data for display, and renders the storefront shell. In practice, 'main.jsx' is the bootstrapping entry and 'App.jsx' is the functional entry for the product experience.

The back-end entry is 'backend/index.js'. Express initializes by importing 'express' and 'cors', creating the app with 'express()', assigning the port from 'process.env.PORT || 5174', and registering middleware with 'app.use(cors())' and 'app.use(express.json())'. It defines an in-memory product catalog, exposes 'GET /api/products', and starts the server with 'app.listen(PORT, ...)'. That route is the main catalog endpoint consumed by the React app.

2. Data Flow & API Contract

The data flow starts in 'frontend/src/App.jsx' inside a 'useEffect' hook that runs on initial render. The component calls 'fetch("http://localhost:5174/api/products")', waits for the response, checks 'res.ok', then parses JSON with 'res.json()'.

The backend route in 'backend/index.js' handles 'GET /api/products'. It reads from the local 'products' array and annotates each item before returning it. The server-side annotations include:

- 'priceFormatted', generated with 'Intl.NumberFormat("en-IN", { style: "currency", currency: product.currency })'
- preserved delivery text from each product's 'delivery' field
- preserved merchandising metadata such as 'badge', 'category', and 'description'
- a top-level 'banner' string returned alongside the curated product list

The API contract returned to the client is:

```
""json
{
  "banner": "Festival Drop - Free COD - Express delivery",
  "curated": [
    {
```

```

    "id": 1,
    "name": "Anaya Chiffon Saree",
    "price": 4599,
    "currency": "INR",
    "badge": "New",
    "category": "Ethnic",
    "description": "Lightweight chiffon saree paired with subtle embroidery.",
    "delivery": "2-3 days",
    "priceFormatted": "Rs. 4,599.00"
  }
]
}
""

```

On the client, the fetched payload populates React state as follows:

- 'setBanner(data.banner)'
- 'setProducts(data.curated)'
- 'setStatus("ready")'

If the request fails or the response is not OK, the app logs the error in the browser console and sets 'status' to '"error"'. The UI uses 'status' in multiple places:

- hero card: shows loading, error, or the curated item count
- product section: shows a fallback prompt if the backend is unavailable
- product grid: conditionally renders loading text or the mapped 'ProductCard' components

This makes readiness visible both in state and in the rendered storefront.

3. Layer Separation

The dependency boundary is cleanly split between rendering concerns in the frontend and API concerns in the backend.

'frontend/package.json' contains rendering and developer-experience packages:

- 'react' and 'react-dom': component rendering and DOM mounting
- 'vite': dev server, module graph, and production build pipeline
- '@vitejs/plugin-react': React support in Vite
- 'eslint', '@eslint/js', 'eslint-plugin-react-hooks', 'eslint-plugin-react-refresh', and 'globals': linting and code-quality tooling
- '@types/react' and '@types/react-dom': editor and tooling support

'backend/package.json' contains API/runtime packages:

- 'express': HTTP server and route definition
- 'cors': cross-origin access for the frontend dev server

If the project grows, the extension points are straightforward:

- database layer: add packages such as 'pg', 'mongoose', or 'prisma' on the backend only
- feature flags or config: add backend config libraries like 'dotenv' and possibly a shared config contract
- client data caching: add frontend libraries such as TanStack Query if API calls become more complex
- validation: add 'zod', 'joi', or 'express-validator' on the backend for response and request contracts
- observability: add backend logging/metrics packages and frontend monitoring only where user telemetry is needed

The current setup keeps rendering logic in the Vite/React layer and data-serving logic in Express, which is a good baseline for scaling.

4. Operational Notes

The backend currently has lightweight operational behavior:

- logging: a startup log is emitted when 'app.listen(...)' succeeds, showing 'http://localhost:\${PORT}'
- error handling: there is no centralized Express error middleware yet
- route failure handling: the '/api/products' route is synchronous and simple, so there is currently no explicit try/catch path
- CORS policy: 'app.use(cors())' enables permissive cross-origin access so the Vite frontend can call the API during development
- JSON parsing: 'app.use(express.json())' is enabled even though the current route is a 'GET', which prepares the app for future JSON request bodies

For onboarding in a repo three times larger, I would describe this backend as:

"The API is a small Express service with a single catalog endpoint, permissive dev CORS, and minimal operational scaffolding. Startup logging exists, but structured request logging, centralized error middleware, validation, and environment-specific CORS controls would be the first improvements as the service surface grows."

That framing gives a new engineer both the current state and the expected next steps without overstating maturity.

5. Summarization Habit

One repeatable process for reading a larger repo is:

1. Start with runtime files: 'package.json', entry points, and start/build scripts.
2. Record the boot sequence for each app: browser entry, root component, server entry, registered routes, and default ports.
3. List the key directories by responsibility, such as 'frontend/src' for UI, 'backend' for APIs, and 'docs' for project guidance.
4. Trace one user-visible flow end to end, such as a page load calling an API and rendering the response.

5. Capture operational notes separately: required commands, ports, environment variables, logging, and obvious scaling gaps.

For this repository, the actionable summary would include:

- frontend run command: 'npm run dev'
- backend run command: 'npm start'
- frontend URL: 'http://localhost:5173'
- backend URL: 'http://localhost:5174'
- primary frontend files: 'frontend/src/main.jsx', 'frontend/src/App.jsx'
- primary backend file: 'backend/index.js'
- primary API: 'GET /api/products'

This habit keeps summaries technical, concise, and useful for the next engineer who needs to run, debug, or extend the codebase quickly.